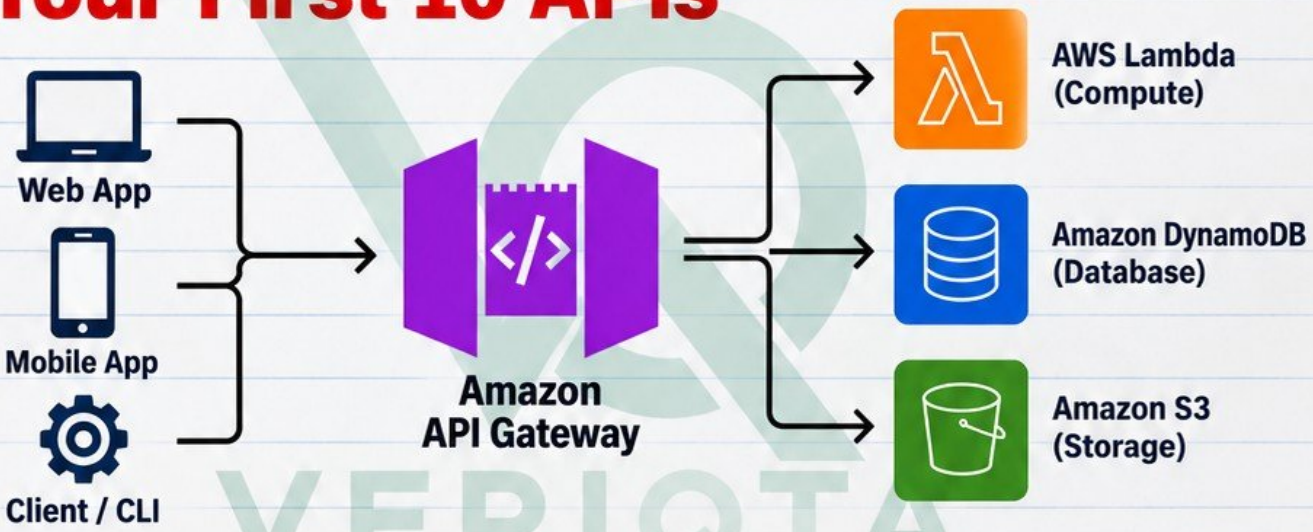
VERIQTA

AWS API Gateway

for Beginners

Build, Secure, and Troubleshoot Your First 10 APIs



VERIQTA

- Build 10 Real APIs
- Secure Your APIs
- Troubleshoot Common Issues
- Monitor and Maintain
- Real-World Examples

- ✅ Step-by-step hands-on examples
- ✅ Authentication and authorization
- ✅ Throttling, API keys and security best practices
- ✅ Error handling and troubleshooting
- ✅ Complete, practical projects

BUILD 10 APIS:

- ✓ FROM FIRST REQUEST TO PRODUCTION-READY
- ✓ REAL EXAMPLES YOU CAN USE

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

VERIQTA

AWS

The 10 APIs Learners Will Build

API	Endpoint	Primary engineering skill
01	GET /health	First integration
02	GET /greetings	Query validation
03	POST /tasks	JSON and DynamoDB
04	GET /tasks/{taskId}	Path parameters
05	GET /tasks	Pagination
06	PATCH /tasks/{taskId}	Safe updates
07	DELETE /tasks/{taskId}	Destructive operations
08	GET /profile	JWT authentication
09	GET /admin/reports	Authorization
10	POST /feedback	Throttling and idempotency

The Journey of an API Request

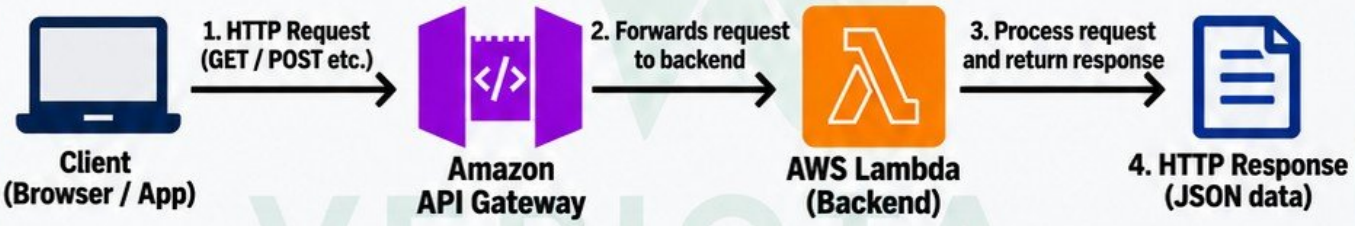
1. What is an API?

- An API (Application Programming Interface) allows different software applications to communicate with each other.
- It defines how a client can request data or perform an action and how the server responds.
- Applications use APIs to access data, integrate services, and build new features without rebuilding everything from scratch.

2. Why Applications Need APIs?

- Enable communication between frontend, backend and third-party services.
- Allow access to data and functionality in a controlled and secure way.
- Power mobile apps, web apps, and integrations with external services (such as payment, email, maps, etc).
- Reduce development time and make systems more scalable.

3. How an API Request Works



4. Role of API Gateway

- Acts as a controlled entry point for your APIs.
- Receives and routes client requests to the correct backend (e.g. Lambda, HTTP service).
- Handles security, authentication, throttling, request validation, and monitoring.
- Hides your backend from direct public access.
- Helps you build, secure, and scale APIs easily on AWS.

5. Common HTTP Methods

Method	What it does
GET	Retrieve data (read)
POST	Create new data
PUT	Replace existing data (full update)
PATCH	Partially update existing data
DELETE	Remove data

6. Common Status Codes

Status Code	Meaning
200	OK (successful request)
201	Created (resource created)
400	Bad Request (invalid input)
401	Unauthorized (missing or invalid credentials)
403	Forbidden (not allowed)
404	Not Found (resource does not exist)
429	Too Many Requests (rate limited)
500	Internal Server Error (backend failure)
502	Bad Gateway (invalid response from backend)

7. Practice This

1. Open a browser or use curl.
2. Send a GET request to your API endpoint (e.g. GET /health).
3. Identify which component is handling each step:
   - Client (your browser or app)
   - API Gateway (receives and routes request)
   - Lambda (processes the request)
   - Response (returned to the client)

8. Troubleshooting Insight

Why a reachable API can still return an error

- The route or method might not exist (404).
- Missing or invalid authentication (401 or 403).
- Invalid request body or parameters (400).
- Backend (e.g. Lambda) might be failing (500 or 502).
- Rate limiting might be enabled (429).
- The API Gateway integration might be misconfigured.

Set Up AWS Without Creating a Security Problem

Prepare a safe environment for building ten APIs.

1. Key AWS Components You Need

AWS Account
Your gateway to all AWS services.

Region
Choose a region close to your users.

IAM Identity
Use an IAM user or IAM role (not root).

AWS CLI
Manage your resources from the command line.

2. Do Not Use Root Credentials



- Full access to everything.
- Cannot be restricted with permissions.
- High risk if credentials are exposed.
- Use an IAM user or IAM role for daily engineering work.
- Keep root account only for account recovery and critical administrative tasks.

3. Configure AWS CLI Access

1. Create an IAM user with programmatic access.
2. Attach appropriate permissions (least privilege).
3. Generate an access key and secret key.
4. Configure the AWS CLI.

```
aws configure
AWS Access Key ID [None]: YOUR_ACCESS_KEY
AWS Secret Access Key [None]: YOUR_SECRET_KEY
Default region name [None]: us-east-1
Default output format [None]: json
```

4. Verify Your AWS Identity

Run this command:

```
> aws sts get-caller-identity
```

Example output:

```
{
  "UserId": "AIDAEXAMPLE123",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/ann"
}
```

This shows the IAM identity you are using, your AWS account ID, and the ARN.

5. Set an AWS Budget



- Helps you monitor and control costs.
- Set a monthly budget and enable alerts.
- Budgets are not hard spending limits.
- You can still incur charges if the budget is exceeded.
- Useful for learning and testing while avoiding unexpected bills.

6. Create a Naming Convention for TaskFlow Resources

Use a clear and consistent naming pattern.

Example:



- Project: taskflow
- Environment: dev | prod
- Resource type: lambda, apigw, table, etc.
- Format: taskflow-dev-lambda-hello
- Use lowercase letters, numbers, and hyphens.

7. Understand Basic AWS Charges

AWS Lambda
- Pay per request and compute time (duration).
- Free tier available.

API Gateway
- Pay per API request.
- HTTP APIs are generally cheaper than REST APIs.
- Free tier available.

CloudWatch
- Pay for log storage, metrics, and alarms.
- Costs depend on data volume.
- Set log retention to manage costs.

8. Practice This

1. Verify your current AWS identity.
2. Check the configured region.
3. Create a test S3 bucket or list existing buckets to confirm access.
4. Review your AWS Budget in the console.

9. Troubleshooting Common Issues

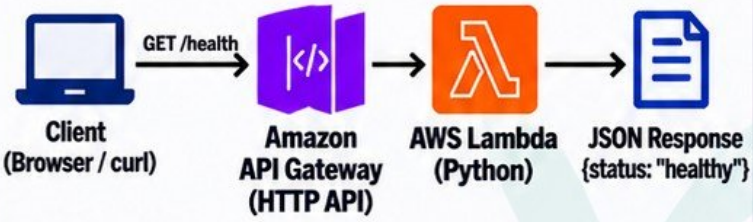
Issue	Possible Cause	Solution
AccessDenied	Missing permissions	Attach the correct IAM policy
Missing credentials	CLI not configured or expired keys	Run aws configure and check keys
Incorrect Region	Using the wrong region	Set the correct region in AWS CLI or command

API 01: Build a Working Health Check

Endpoint: GET /health

Deploy the first public HTTP endpoint using API Gateway and Lambda.

1. Architecture Overview



2. Create a Python Lambda Function

```python
import json

def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({"status": "healthy"})
    }
```

3. Create an API Gateway HTTP API

1. Go to API Gateway → Create API.
2. Choose HTTP API.
3. Name it (e.g. taskflow-api).
4. Keep the default stage ($default).
5. Create the API.

Amazon API Gateway HTTP API

4. Connect the GET Route to Lambda

1. In your API, click Create Route.
2. Select GET and enter /health.
3. Choose Lambda as the integration.
4. Select your Lambda function.
5. Create the route and deploy (auto-deploys to $default stage).

5. Understand the Invoke URL

- After creation, AWS provides an invoke URL.
- The default stage for HTTP APIs is $default.
- The URL looks like:

  `https://{api-id}.execute-api.{region}.amazonaws.com`

- Your endpoint will be:

  `https://{api-id}.execute-api.{region}.amazonaws.com/health`

6. Test with curl

```
> curl -i https://{api-id}.execute-api.
  {region}.amazonaws.com/health
```

Example response:

```
HTTP/2 200
content-type: application/json
content-length: 22

{"status"::"healthy"}
```

7. Troubleshoot Common Issues

Issue	Possible Cause	Solution
404 Not Found	Route not created or wrong path	Check the route is GET /health
500 Internal Error	Lambda function error	Check Lambda logs in CloudWatch
403 Forbidden	Missing Lambda invocation permission	Ensure API Gateway has permission to invoke the function
No response or timeout	Incorrect integration configuration	Verify integration type and function

8. Verify the Result

You should receive:

`{"status":"healthy"}`

with HTTP 200 status code.

9. Production Insight

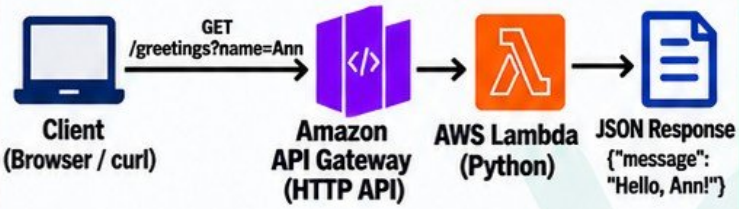
A basic health response does not prove that downstream dependencies are healthy (e.g. databases, external services). In production, include deeper health checks if needed.

API 02: Accept Input Without Trusting It

Endpoint: GET /greetings?name=Ann

Understand query parameters and safe input handling.

1. How Query Parameters Work



2. Read Query Parameters in Lambda

```python
def lambda_handler(event, context):
    params = event.get("queryStringParameters")
    name = params.get("name") if params else None

    if not name:
        return {
            "statusCode": 400,
            "body": json.dumps({
                "message": "Missing name parameter"
            })
```

3. Validate the Input

1. Check if the name parameter exists.
2. Validate that it is not empty.
3. Set a reasonable length limit (e.g. 1–50 characters).
4. Return HTTP 400 for invalid input.
5. Sanitize input to avoid unexpected issues.

4. Complete Lambda Function Example

```python
import json
def lambda_handler(event, context):
    params = event.get("queryStringParameters") or {}
    name = params.get("name")
    if not name:
        return { "statusCode": 400,
         "body": json.dumps({"message": "Missing name parameter"})
    }
    if len(name.strip()) == 0:
        return { "statusCode": 400,
         "body": json.dumps({"message":"Name cannot be empty"})
    }
    if len(name) > 50:
        return { "statusCode": 400,
         "body": json.dumps({"message": "Name is too long"})
    }
    return { "statusCode": 200,
        "body": json.dumps({"message": f"Hello, {name}!"})
    }
```

5. Test with curl

Valid request:

```
> curl -i "https://{api-id}.execute-api.
  {region}.amazonaws.com/greetings?name=Ann"
```

Response:

```
HTTP/1.1 200 OK
content-type: application/json
{"message":"Hello, Ann!"}
```

Invalid request (missing name):

```
> curl -i "https://{api-id}.execute-api.
  {region}.amazonaws.com/greetings"
```

Response:

```
HTTP/1.1 400 Bad Request
content-type: application/json
{"message":"Missing name parameter"}
```

6. Understand Errors

- 400 (Bad Request): Client sent invalid input.
- 500 (Internal Server Error): Something went wrong in your Lambda code.
- Distinguish between client errors (fix the request) and backend failures (check logs).

7. Troubleshooting Common Issues

Issue	Possible Cause	Solution
Parameter is None	Query parameter missing	Check event structure and use .get()
Lambda exception	Unexpected event structure	Print the event and verify the correct key
Always returns 400	Validation logic too strict	Review validation rules
Returns 500	Code error	Check CloudWatch logs

8. Verify the Result

- Valid input produces a greeting.
- Invalid input produces a useful error message with HTTP 400.

9. Production Insight

Always validate and sanitize user input. Do not trust query parameters, headers, or request data, even if the source seems trusted. Invalid or malicious input can cause errors or unexpected behavior.

API 03: Create a Task With POST

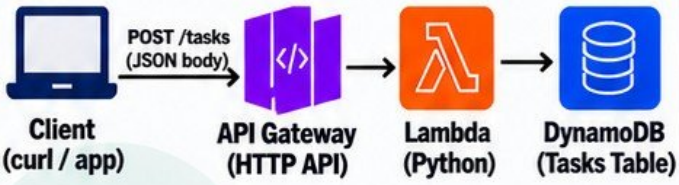
Endpoint: POST /tasks

Accept JSON request bodies and create a resource.

1. GET vs POST and Why It Matters

GET	POST
Used to retrieve data.	Used to create new data.
Does not change data.	Sends data in the request body (JSON).
Data sent in the URL (query parameters).	Can change data.
Idempotent and safe to repeat.	Not necessarily idempotent.

2. Architecture Overview



3. Request Format and Content Type

- Use Content-Type: application/json in the request header.
- Send a valid JSON body with the required fields.

```
curl -X POST "https://{api-id}.execute-api.
  {region}.amazonaws.com/tasks" \
   -H "Content-Type: application/json" \
   -d '{
      "title": "Learn AWS",
      "status": "pending"
   }'
```

4. Lambda Function (Python)

```
import json
import uuid
import boto3

dynamodb = boto3.reource('dynamodb')
table = dynamodb.Table('taskflow-tasks")

def lambda_handler(event, context):
    try:
        body = json.loads(event['body'])
        title = body.get("title")
        status = body.get('status', "pending')

        if not title:
            return {"statusCode": 400,
                    "body": json.dumps({"message": "Title is required"})}

        task_id = str(uuid.uuid4())
        item = {"id": task_id, "title": title, "status": status}
        table.put_item(Item=item)

        return {"statusCode": 201,
                "body": json.dumps(item)}

    except json.JSONDecodeError:
        return {"statusCode": 400,
                "body": json.dumps({"message": "Invalid JSON body"})}
```

5. Validate the Request Body

1. Parse the JSON body from the event.
2. Check that required fields (title) are present.
3. Validate data types and length (e.g. title not empty and reasonable length).
4. Return HTTP 400 for invalid input.

6. Store the Task in DynamoDB

Amazon DynamoDB

- Create a DynamoDB table (e.g. taskflow-tasks).
- Store the task with an ID, title, and status.
- Use appropriate partition key (e.g. id).

7. Give Lambda Narrowly Scoped Permissions

IAM Role

- Attach an IAM role to the Lambda function.
- Allow only required DynamoDB actions (e.g. dynamodb:PutItem on the specific table).
- Do not use overly broad permissions.

8. Successful Response (HTTP 201)

Response:

```
HTTP/1.1 201 Created
Content-Type: application/json
{
  "id": "e3f2a9c4-5b6d-4e8f-9c1a-123456789abc",
  "title": "Learn AWS",
} "status": "pending"
```

9. Troubleshooting Common Issues

Issue	Possible Cause	Solution
400 Bad Request (Invalid JSON)	Malformed JSON body	Check JSON format and Content-Type header
400 Missing Field	title is missing or empty	Validate input and provide a clear error
403/AccessDenied	Lambda role does not have DynamoDB permissions	Attach correct IAM policy with least privilege
500 Internal Error	Unhandled exception in Lambda	Check CloudWatch logs for details

10. Verify the Result

- A task exists in DynamoDB after the request.
- The item contains an ID, title, and status.
- Status code is 201.

API 04: Retrieve a Task by ID

Endpoint: GET /tasks/{taskId}

Retrieve a specific resource and understand path parameters.

1. Path Parameters vs Query Parameters

Path Parameters

- Part of the URL path.
- Identify a specific resource.
- Used when you need a unique ID.
- Example:

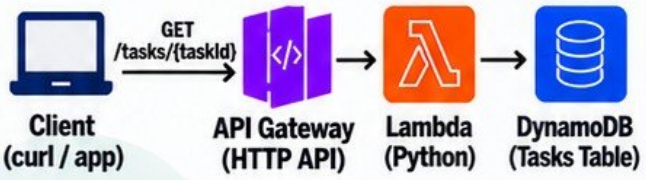
`/tasks/{taskId}`

Query Parameters

- Added after `?` in the URL.
- Used for optional filters or additional data.
- Used when you are not targeting a specific resource.
- Example:

`/tasks?status=completed`

2. Architecture Overview



3. Read Path Parameters in Lambda

```python
import json
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('taskflow-tasks')

def lambda_handler(event, context):
    task_id = event.get('pathParameters', {}).get('taskId')
    if not task_id:
        return {
            "statusCode": 400,
            "body": json.dumps({ "message": "Task ID is required"})
    }
```

4. Retrieve a Task from DynamoDB

```python
response = table.get_item(Key={"id": task_id})
item = response.get('Item')

if not item:
    return {
        "statusCode": 404,
        "body": json.dumps({"message": "Task not found"})
    }

return {
    "statusCode": 200,
    "body": json.dumps(item)
}
```

5. Test with curl

Valid request:

```
> curl -i "https://{api-id}.execute-api.
  {region}.amazonaws.com/tasks/e3f2a9c4"
```

Response (HTTP 200):

```
HTTP/1.1 200 OK
Content-Type: application/json
{
   "id": "e3f2a9c4",
   "title": "Learn AWS",
   "status": "pending"
}
```

Nonexistent task:

```
> curl -i "https://{api-id}.execute-api.
  {region}.amazonaws.com/tasks/9999"
```

Response (HTTP 404):

```
HTTP/1.1 404 Not Found
Content-Type: application/json
{
   "message": "Task not found"
}
```

6. Common Errors and How to Fix Them

Issue	Possible Cause	Solution
400 Bad Request	Missing or empty taskId	Check the URL and ensure the path parameter is provided.
404 Not Found	Task does not exist in the table	Verify the task ID or create the task first.
AccessDenied	Lambda role lacks DynamoDB permissions	Attach an IAM policy with dynamodB:GetItem on the specific table.
500 Internal Error	Incorrect table name or runtime error	Check the table name, code and CloudWatch logs.

7. Practice This

1. Retrieve an existing task using its ID.
2. Try a nonexistent task ID.
3. Check the Lambda logs for each request.
4. Confirm that the correct status codes are returned.

8. Verify the Result

- The correct task is returned for a valid task ID.
- A 404 response is returned for a nonexistent task.
- No internal errors or sensitive information are exposed.

9. Production Insight

A "not found" response should not expose internal errors or database details. Return a clear and consistent error message to the client.

API 05: List Tasks With Controlled Results

Endpoint: GET /tasks

Build a list endpoint without creating an expensive database operation.

1. What is a Collection Endpoint?

- Returns multiple resources (a list).
- Used to retrieve tasks for a user or project.
- Generally supports pagination, filtering and sorting.
- Endpoint: GET /tasks

2. Why You Should Not Return Every Record

- Can be slow and expensive.
- Consumes high read capacity in DynamoDB.
- Increases latency for users.
- Can expose too much data.
- Does not scale as data grows.

3. DynamoDB Query vs Scan

Query (Recommended)	Scan (Avoid for large tables)
Efficient and fast.	Reads every record in the table.
Uses a partition key (and optional sort key).	Slow and expensive.
Returns only relevant data.	High read capacity cost.
Lower read capacity cost.	Not suitable for production list endpoints.

4. Example Table Design (Access Pattern)

Partition Key (PK)	Sort Key (SK)	Example Value
userId (string)	taskId (string)	user-123 task-001

- Stores tasks by user.
- Allows efficient query of a user's tasks.
- Supports pagination using the sort key (e.g. createdAt).

5. Lambda Function (Python - DynamoDB Query)

```python
import json
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('taskflow-tasks')

def lambda_handler(event, context):
    params = event.get('queryStringParameters') or {}
    user_id = params.get('userId')
    limit = int(params.get('limit', 10))
    next_token = params.get('nextToken')

    if not user_id:
        return {"statusCode": 400, "body": json.dumps({"message": "userId is required"})}

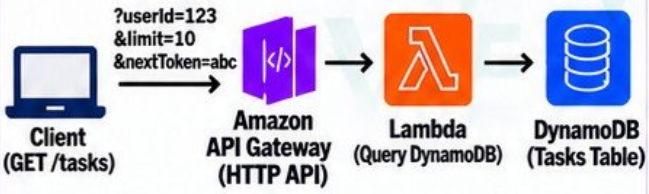
    query_params = {
        'KeyConditionExpression': 'userId = :uid',
        'ExpressionAttributeValues': {':uid': user_id},
        'Limit': limit
    }
    if next_token:
        query_params['ExclusiveStartKey'] = json.loads(next_token)

    response = table.query(**query_params)
    items = response.get('Items', [])
    next_key = response.get('LastEvaluatedKey')

    return {
        'statusCode': 200,
        'body': json.dumps({
            'tasks': items,
            'nextToken': json.dumps(next_key) if next_key else None
        })
    }
```

6. Understanding Pagination

- Use a limit parameter to control the number of results.
- Return a continuation token (nextToken) when more data is available.
- Client sends the next Token to get the next page.
- Prevents large responses and reduces cost.
- Handle invalid or malformed nextToken values.



7. Test with curl

First page (limit 5):

```
curl -i "https://{api-id}.execute-api.{region}.amazonaws.com/tasks?userId=user-123&limit=5"
```

Next page using nextToken:

```
curl -i "https://{api-id}.execute-api.{region}.amazonaws.com/tasks?userId=user-123&limit=5&nextToken={token}"
```

8. Example Success Response (HTTP 200)

```
{
  "tasks": [
    {"id": "task-001", "title": "Learn AWS", "status": "pending"},
    {"id": "task-002", "title": "Build API", "status": "completed"}
  ],
  "nextToken": "{\"userId\":{\"S\":\"user-123\"},...}"
}
```

9. Handle Empty Results

```
{
  "tasks": [],
  "nextToken": null
}
```

Return an empty list with HTTP 200 when no tasks are found.

10. Troubleshooting Common Issues

Issue	Possible Cause	Solution
Missing records	Incorrect query key or access pattern	Verify partition key and sort key values
Repeated pages	Not using nextToken correctly	Use ExclusiveStartKey from the previous response
Validation error on nextToken	Malformed token	Ensure the token is valid JSON and not modified
AccessDenied	Missing DynamoDB permissions	Attach IAM policy with dynamodb:Query on the specific table

11. Production Insight

Your database access pattern should influence your API design. Design your DynamoDB keys and query patterns around how your application needs to access the data, not the other way around.

API 06: Update a Task Safely

Endpoint: PATCH /tasks/{taskId}

Modify existing resources without accidentally overwriting data.

1. PUT vs PATCH and Their Semantics

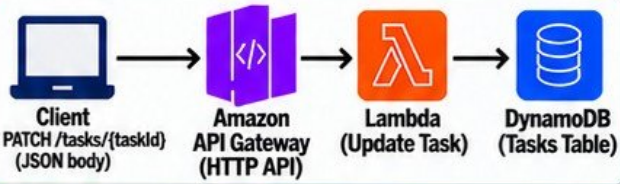
PUT (Replace)

- Replaces the entire resource with the provided data.
- All fields must be provided.
- Used when you want to update everything.
- Can overwrite existing data if fields are missing.

PATCH (Partial Update)

- Updates only the specified fields.
- Used for partial changes (e.g. just the status).
- Reduces the risk of overwriting data.
- Preferred for this endpoint since we only update specific fields.

2. Request Flow



3. Allowed Task Status Values

Validate the status field to ensure only allowed values are used.

`pending` `in-progress` `completed`

4. Lambda Function (Python - Conditional Update)

```python
import json
import boto3
from botocore.exceptions import ClientError

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('taskflow-tasks')

ALLOWED_STATUS = ['pending', 'in-progress', 'completed']

def lambda_handler(event, context):
    task_id = event.get('pathParameters', {}).get('taskId')
    body = json.loads(event.get('body', '{}'))
    status = body.get('status')

    if not status or status not in ALLOWED_STATUS:
        return {
            'statusCode': 400,
            'body': json.dumps({"message": 'Invalid or missing status"})
        }
    try:
        response = table.update_item(
            Key={'id': task_id},
            UpdateExpression="SET #s = :status, updatedAt = :now, version = version + :one",
            ConditionExpression="attribute_exists(id)",
            ExpressionAttributeNames={'#s': 'status'},
            ExpressionAttributeValues={
                ':status': status,
                ':now': boto3.dynamodb.types.TypeSerializer().serialize(
                    int(__import__('time').time())
                ),
                ':one': {'N': '1'}
            },
            ReturnValues='ALL_NEW'
        )
        return {
            'statusCode': 200,
            'body': json.dumps(response.get('Attributes'))
        }
    except ClientError as e:
        if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
            return {
                'statusCode': 404,
                'body': json.dumps({"message": 'Task not found"})
            }
        return {
            'statusCode': 500,
            'body': json.dumps({"message":'Database error"})
        }
```

5. Test with curl

Valid update (pending → completed):

```
curl -X PATCH "https://{api-id}.execute-api.
 {region}.amazonaws.com/tasks/task-001"  \
 -H "Content-Type: application/json" \
 -d '{"status": "completed"}'
```

Invalid status value:

```
curl -X PATCH "https://{api-id}.execute-api.
 {region}.amazonaws.com/tasks/task-001"  \
 -H "Content-Type: application/json" \
 -d '{"status": "cancelled"}'
```

6. Example Success Response (HTTP 200)

```
{
   "id": "task-001",
   "title": "Learn AWS",
   "status": "completed",
   "createdAt": 1698880000,
   "updatedAt": 1698881234,
   "version": 2
}
```

7. Troubleshooting Common Issues

Issue	Possible Cause	Solution
400 Bad Request	Invalid or missing status value	Check request body and allowed values
404 Not Found	Task does not exist (conditional update failed)	Verify the task ID
500 Internal Error	Incorrect table name or other database error	Check table name, IAM permissions and logs
Validation error	Malformed JSON body	Ensure valid JSON and Content-Type header

8. Practice This

1. Update a task from pending to completed.
2. Try updating with an invalid status.
3. Try updating a nonexistent task.
4. Check CloudWatch logs for errors.
5. Use a second GET request to confirm the update.

9. Verify the Result

- A second GET /tasks/{taskId} returns the updated task.
- Only the specified fields are changed.
- No unintended data is overwritten.

10. Production Insight

Use conditional updates and versioning to prevent lost updates. Always validate input and return clear error messages without exposing internal system details.

API 07: Delete a Task Without Deleting the Wrong One

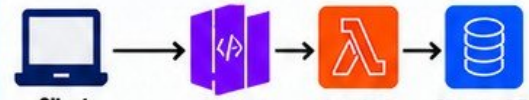
Endpoint: DELETE /tasks/{taskId}

Handle destructive API operations responsibly.

1. What DELETE Does

- DELETE removes a resource from the database.
- It is a destructive operation.
- Authorization is important to prevent accidental or malicious deletion.
- Ensure the user is allowed to delete the task (e.g. owner).

2. Request Flow



3. Validate the Task ID

- Check that the taskId is present and valid.
- Use the correct format (e.g. UUID).
- Return 400 for missing or invalid taskId.

4. Lambda Function (Python - Conditional Delete)

```python
import json
import boto3
from botocore.exceptions import ClientError

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('taskflow-tasks')

def lambda_handler(event, context):
    task_id = event.get('pathParameters', {}).get('taskId')
    if not task_id:
        return {
          'statusCode': 400,
          'body': json.dumps({"message": 'Task ID is required'})
        }
    try:
        # Conditional delete - only delete if the item exists
        table.delete_item(
            Key={'id': task_id},
            ConditionExpression='attribute_exists(id)'
        # Return 204 No Content
        return {
            'statusCode': 204,
        }  'body': '''
    except ClientError as e:
        if e.response['Error']['Code']== 'ConditionalCheckFailedException':
            return {
              'statusCode': 404,
              'body': json.dumps({"message": 'Task not found"})
        return {
            'statusCode': 500,
        }  'body': json.dumps({"message": 'Database error"})
```

5. Status Codes

Status Code	Meaning
204	**No Content** Task deleted successfully. No response body.
404	**Not Found** Task does not exist (or user is not authorized).
400	**Bad Request** Missing or invalid taskId.
403	**Forbidden** User is not allowed to delete this task.
500	**Internal Server Error** Unexpected error in the Lambda function.

6. Test with curl

Valid delete request (existing task):

```
> curl -i -X DELETE "https://{api-id}.execute-api.{region}.amazonaws.com/tasks/task-001"
```

Response:

```
HTTP/1.1 204 No Content
```

Delete a nonexistent task:

```
> curl -i -X DELETE "https://{api-id}.execute-api.{region}.amazonaws.com/tasks/9999"
```

Response:

```
HTTP/1.1 404 Not Found
Content-Type: application/json
{"message":"Task not found"}
```

7. Idempotency and Repeat DELETE Requests

- DELETE is idempotent. If you delete the same resource multiple times, the result should be the same.
- The first request returns 204 (if the item exists).
- Subsequent requests should return 404 (resource no longer exists).
- Design your API so repeated DELETE requests do not cause errors or unintended side effects.

8. Troubleshooting Common Issues

Issue	Possible Cause	Solution
Task not deleted	Wrong taskId or item does not exist	Verify the task ID and check in DynamoDB
AccessDenied	Lambda role missing DynamoDB permissions	Attach IAM policy with dynamodb:DeleteItem
Unexpected deletion	Missing ownership check	Ensure user can only delete their own tasks
Validation error (400)	Missing or invalid taskId	Check request path and ID format

9. Practice This

1. Delete a test task that exists.
2. Try to delete the same task again.
3. Attempt to delete a nonexistent task.
4. Check the Lambda logs for each request.
5. Confirm the correct status codes are returned.

10. Production Insight

- Auditability matters for destructive operations.
- Consider logging all delete requests (user, taskId, timestamp).
- In some systems, soft delete (mark as deleted) may be preferred.
- Have a recovery strategy for accidental deletions.

API 08: Protect a User Profile With JWT Authentication

Endpoint: GET /profile

Stop treating possession of a URL as permission to access data.

1. Authentication vs Authorization

Authentication

- Proves who you are.
- Usually done with a username and password (or a token).
- In this project we use Amazon Cognito and JWTs.

Authorization

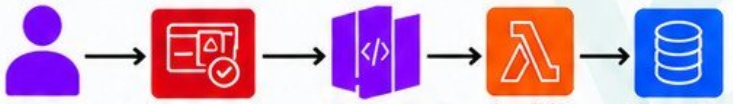
- Decides what you are allowed to access.
- Enforced by your API and backend code.
- A valid token does not automatically mean you can access all data.

2. What is a JSON Web Token (JWT)?

- A signed token that contains claims about the user.
- Made of three parts: header, payload and signature.
- Used to authenticate requests to your API.
- Contains information like user ID, expiry time (exp), and scopes.

Header	Payload	Signature
Algorithm and token type (e.g. HS256)	Claims such as user ID, issuer (iss), audience (aud), expiry (exp) and scopes.	Verifies that the token has not been modified.

3. Architectiure Overview



4. Configure an HTTP API JWT Authorizer

1. Create a Cognito User Pool and App Client.
2. In API Gateway (HTTP API), create a JWT authorizer.
3. Configure the issuer (iss) from your Cognito User Pool.
4. Add the audience (aud) – your App Client ID.
5. Set route-level authorization scopes (if using scopes).

5. Lambda Function (Python - Get User Profile)

```python
import json
import boto3
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('taskflow-users')

def lambda_handler(event, context):
    # User information from Cognito JWT (added by API Gateway)
    claims = event.get('requestContext', {}).get('authorizer', {}).get('jwt', {}).get('claims',{})
    user_id = claims.get('sub')  # Cognito user ID

    if not user_id:
        return {
            'statusCode': 401,
            'body': json.dumps({"message": 'Unauthorized'})
    }
    # Get only the current user's profile
    response = table.get_item(Key={'id': user_id})
    item = response.get('Item')
    if not item:
        return {
            'statusCode': 404,
            'body': json.dumps({"message": 'Profile not found'})
    }
    return {
        'statusCode': 200,
        'body': json.dumps({"profile": item})
}
```

6. Test with curl

Missing token (401):

```
> curl -i "https://{api-id}.execute-api.
{region}.amazonaws.com/profile"
```

Response:

```
HTTP/1.1 401 Unauthorized
{ "message": "Unauthorized" }
```

Expired or invalid token (401):

```
> curl -i -H "Authorization: Bearer invalidtoken" \
"https://{api-id}.execute-api.{region}.amazonaws.com/profile"
```

Response:

```
HTTP/1.1 401 Unauthorized
{ "message": "Invalid token" }
```

Valid token (200):

```
> curl -i -H "Authorization: Bearer {valid-token}" \
"https://{api-id}.execute-api.{region}.amazonaws.com/profile"
```

Response:

```
HTTP/1.1 200 OK
{ "profile": { "id": "user-123", "name": "Ann",
   "email": "ann@example.com" } }
```

7. Important JWT Claims to Validate

Claim	What It Means	Why It Matters
iss	Issuer (Cognito User Pool URL)	Ensures token is from your user pool.
aud	Audience (App Client ID)	Ensures token is for your application.
exp	Expiry time	Reject expired tokens.
scope	Authorization scopes	Ensures user has required permissions.
sub	User ID	Used to retrieve the correct user's data.

8. Troubleshooting Common Issues

Issue	Possible Cause	Solution
401 Unuathorized	Missing, expired or invalid token	Check token format, expiry and issuer.
403 Forbidden	Valid token but missing required scope	Add correct scopes in token and in API Gateway route.
Incorrect claims	Wrong issuer or audience	Use the correct Cognito User Pool ARN and App Client ID.
User sees another user's data	Backend does not enforce user-specific access	Use the user ID from the token (sub) to query only that user's data.

9. Practice This

1. Sign up a user in Cognito and get a JWT.
2. Call GET /profile with a valid token.
3. Test with a missing token, expired token, and invalid token.
4. Verify that the response only contains the authenticated user's profile.

10. Production Insight

- API Gateway HTTP APIs support JWT authorizers and route-level authorization scopes.
- Always enforce user-specific data access in your backend.
- Do not rely only on the token for authorization.
- Auditability and monitoring are important for secured APIs.

API 09: Build an Admin-Only Endpoint

Endpoint: GET /admin/reports

Learn why being logged in does not mean having every permission.

1. User Identity vs Permission

User Identity (Who you are)

- Provided by authentication (e.g. Cognito JWT).
- Contains claims like: sub (user ID), email, username.
- Tells us who the user is.

Permission (What you can do)

- Controlled by roles, scopes or groups.
- Determines which resources or actions you are allowed to access.
- Being logged in does not automatically give you admin permissions.

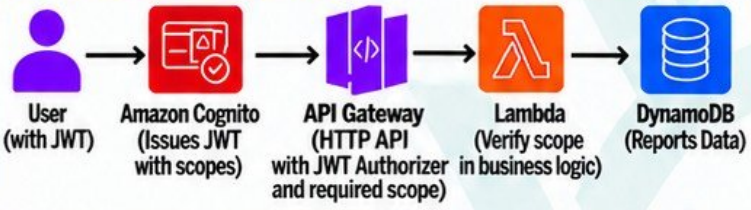
2. Authorization with Scopes

- Use authorization scopes to protect admin endpoints.
- Assign an admin scope (e.g. admin:reports).
- Configure the HTTP API JWT authorizer to require the admin scope.
- Also enforce the scope in your backend business logic (if needed).
- Follow the principle of least privilege.

Example Scopes

read:profile	tasks:read	tasks:write	admin:reports
View own profile	View tasks	Create/modify tasks	Access admin reports

3. Architecture Overview



4. Configure the Admin Endpoint

1. Create or update your Cognito User Pool and App Client.
2. Add a scope for admin access (e.g. admin:reports).
3. Configure the HTTP API JWT authorizer with the User Pool issuer and audience.
4. Set the required authorization scope on the GET /admin/reports route.
5. In your Lambda function, verify the user has the admin scope before returning the data.

5. Lambda Function (Python - Check Admin Scope)

```python
import json
from botocore.exceptions import ClientError

def lambda_handler(event, context):
    # JWT claims are passed in requestContext by API Gateway
    claims = event.get('requestContext', {}).get('authorizer', {}).get('jwt', {}).get('claims', {})
    scopes = claims.get('scope', '') # space separated scopes
    user_id = claims.get('sub')

    required_scope = 'admin:reports'
    if required_scope not in scopes.split(():
        return {
            'statusCode': 403,
            'body': json.dumps({'message': 'Insufficient permissions'})
        }

    # TODO: retrieve and return your admin reports (e.g. from DynamoDB)
    reports = [
        {'id': 'r-001', 'totalTasks': 25, 'activeUsers': 10}
    ]

    return {
        'statusCode': 200,
        'body': json.dumps({'reports': reports})
    }
```

6. Test with curl

Regular user (should be 403):

```
> curl -i -H "Authorization: Bearer <user-token>" \
  "https://{api-id}.execute-api.{region}.amazonaws.com/admin/reports"
```

Response:

```
HTTP/1.1 403 Forbidden
Content-Type: application/json
{ "message": "Inaufficient permissions" }
```

Missing or invalid token (should be 401):

```
> curl -i "https://{api-id}.execute-api.{region}.amazonaws.com/admin/reports"
```

Response:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
{ "message": "Unauthorized" }
```

Admin user (should succeed):

```
> curl -i -H "Authorization: Bearer <admin-token>" \
  "https://{api-id}.execute-api.{region}.amazonaws.com/admin/reports"
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json
{ "reports": [ { "id": "r-001", "totalTasks": 25, "activeUsers": 10 } ] }
```

7. Common Error Codes

Status Code	Meaning	Common Cause
401	Unauthenticated	Missing, expired or invalid token. Incorrect issuer or audience.
403	Unauthorized	Token is valid but missing the required scope (e.g. admin:reports).
404	Not Found	Route does not exist or wrong URL.
500	Internal Server Error	Lambda error or incorrect configuration.

8. Troubleshooting Common Issues

Issue	Possible Cause	Solution
403 Forbiddon (for admin)	Missing admin scope	Check token scopes and API Gateway route configuration.
401 Unauthorized	Expired or invalid token	Get a new token and verify issuer and audience.
Incorrect audience	Wrong App Client ID	Use the correct Cognito App Client ID. in the authorizer.
User has no data	Backend logic not filtering by user or scope	Ensure backend enforces scope and user-specific access.

9. Practice This

1. Call the admin endpoint with a normal user token.
2. Call the endpoint with an admin token.
3. Check CloudWatch logs for each request.
4. Try an expired token.
5. Verify only administrators can access the data.

10. Production Insight

- Being authenticatted does not mean a user should have access to every resource.
- Always enforce authorization in your backend business logic.
- Use least privilege and clear authorization scopes.
- Audit and monitor access to sensitive admin endpoints.
- Proper logging and monitoring help detect unauthorized access.

API 10: Protect an API From Excessive Requests

Endpoint: POST /feedback

Handle request bursts, abuse, and duplicate submissions.

1. What We Are Building

- A feedback submission endpoint that is protected from abuse, excessive requests, and duplicate submissions.
- Validate the request payload.
- Configure HTTP API route-level throttling.
- Use idempotency keys to prevent duplicate submissions.
- Handle errors correctly (e.g. 429 Too Many Requests).

2. Architecture Overview



Key Protections

- Input validation
- Route-level throttling (rate limits + burst)
- Idempotency keys (prevent duplicates)
- Proper error handling (e.g. 429)

3. Request Payload and Validation

Example Request (POST /feedback)

```
{
  "name": "Ann",
  "email": "ann@example.com",
  "message": "Great content!",
  "idempotencyKey": "a1b2c3d4-e5f6"
}
```

Validation Rules

- Content-Type must be application/json.
- Required fields: name, email, message.
- Validate email format.
- Enforce maximum payload size (e.g. 10 KB).
- Check idempotencyKey is provided and well formed (UUID).

4. Configure HTTP API Throttling

1. Open your HTTP API in API Gateway.
2. Configure route-level throttling for POST /feedback.
3. Set a reasonable rate limit (e.g. 10 requests/second) and burst limit (e.g. 20).
4. Understand that excessive requests will receive HTTP 429 Too Many Requests.
5. Remember: HTTP API throttling is not the same as REST API usage plans with per-client API keys and quotas.

Example Throttling Configuration

Setting	Value (Example)
Rate limit (steady state)	10 requests/second
Burst limit	20 requests

5. Lambda Function (Python - Feedback Submission)

```python
import json
import boto3
from datetime import datetime
from botocore.exceptions import ClientError

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('feedback')

def lambda_handler(event, context):
    try:
        body = json.loads(event.get('body', '{}'))
        name = body.get('name')
        email = body.get('email')
        message = body.get('message')
        idempotency_key = body.get('idempotencyKey')

        if not all([name, email, message, idempotency_key]):
            return {"statusCode": 400, "body": json.dumps({
                "message": "name, email, message and idempotencyKey are required"
            })}
        # Check if this idempotency key already exists
        existing = table.get_item(Key={'idempotencyKey': idempotency_key})
        if 'Item' in existing:
            return {"statusCode": 200, "body": json.dumps({
                "message": "Duplicate submission ignored",
                "feedbackId": existing['Item']['feedbackId']
            })}
        feedback_id = f"fb-{int(datetime.utcnow().timestamp())}"
        item = {
            'feedbackId': feedback_id,
            'name': name,
            'email': email,
            'message': message,
            'idempotencyKey': idempotency_key,
            'createdAt': datetime.utcnow().isoformat()}
        table.put_item(Item=item)
        return {"statusCode": 201, "body": json.dumps(item)}
    except json.JSONDecodeError:
        return {"statusCode": 400, "body": json.dumps({"message": "Invalid JSON body"})}
    except ClientError as e:
        return {"statusCode": 500, "body": json.dumps({"message": "Database error"})}
```

6. Test with curl

Valid request:

```
> curl -i -X POST "https://{api-id}.execute-api.
  {region}.amazonaws.com/feedback" \
  -H "Content-Type: application/json" \
  -d '{"name":"Ann","email":"ann@example.com",
       "message":"Great!","idempotencyKey":"a1b2c3"}'
```

Response:

```
HTTP/1.1 201 Created
{
    "feedbackId": "fb-1769888000",
    "name": "Ann",
    "message": "Great!",
    "idempotencyKey": "a1b2c3"
}
```

Excessive requests (rate limit exceeded):

```
> curl -i -X POST "https://{api-id}.execute-api.
  {region}.amazonaws.com/feedback"
```

Response:

```
HTTP/1.1 429 Too Many Requests
{
    "message": "Rate limit exceeded"
}
```

7. Troubleshooting Common Issues

Issue	Possible Cause	Solution
429 Too Many Requests	Rate limit or burst limit exceeded	Reduce request rate or increase limits
400 Bad Request	Invalid JSON or missing fields	Check payload and Content-Type
Duplicate submissions	Missing or reused idempotency key	Provide a unique idempotency key
500 Internal Error	DynamoDB permissions or code error	Check IAM permissions and logs
Requests not throttled	Throttling not configured on route	Verify route-level throttling settings

8. Practice This

1. Submit a valid feedback request.
2. Send multiple requests quickly to trigger 429.
3. Try the same request with the same idempotency key.
4. Check CloudWatch logs for each request.
5. Verify that valid submissions are saved and duplicates are ignored.

9. Production Insight

- Use throttling to protect your API from abuse and traffic spikes.
- Idempotency keys prevent duplicate submissions.
- HTTP API throttling is different from REST API usage plans with per-client API keys and quotas.
- Monitor metrics, set alarms, and consider backoff and retries.
- Design for graceful degradation under heavy load.

API Keys, CORS, and the Security Traps Beginners Miss

Focus: Understand three common API security misconceptions.

1. Three Common Misconceptions

1. API Keys Are Not Authentication

- An API key is just an identifier, not a proof of identity.
- API keys help with tracking, throttling, and usage plans.
- Anyone with the key can call the API.
- Never use an API key alone to protect sensitive data.

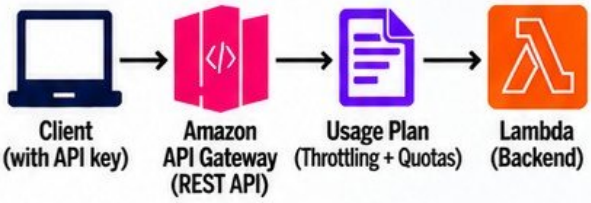
2. CORS Does Not Secure Your API

- CORS only controls which browsers can access your API.
- It is a browser security feature, not an API security control.
- Non-browser clients (e.g. curl, Postman) can still call your API regardless of CORS settings.

3. Authentication, Throttling and Browser Controls Solve Different Problems

- Authentication verifies who you are.
- Throttling limits how much you can do.
- CORS controls browser access.
- Each solves a different problem and should be used together where appropriate.

2. REST API With a Usage Plan and API Key



1. Create a REST API in API Gateway.
2. Enable an API key and create a usage plan.
3. Set throttling and quota limits in the usage plan.
4. Associate the API key with the usage plan.
5. Require the API key on the method (e.g. x-api-key header).
6. Deploy and test the API with a valid and invalid API key.

3. Understanding CORS

- CORS (Cross-Origin Resource Sharing) controls which origins, methods, and headers a browser is allowed to use.
- Browsers send a preflight OPTIONS request for certain requests (e.g. non-GET or custom headers).
- You must configure allowed origins, methods, and headers in API Gateway.
- CORS errors happen in the browser before the request reaches your backend.
- CORS does not stop non-browser clients.

CORS

4. Configure CORS in API Gateway (REST API)

Example CORS Configuration

```
{
    "allowOrigins": ["https://yourdomain.com"],
    "allowMethods": ["GET", "POST", "OPTIONS"],
    "allowHeaders": ["Content-Type", "x-api-key"],
    "allowCredentials": false
}
```

5. Example: Call REST API With an API Key

```
curl -X GET "https://api-id.execute-api.region.amazonaws.com/prod/data" \
    -H "x-api-key: YOUR_API_KEY"
```

Successful Response (valid API key):

```
HTTP/1.1 200 OK
{
   "message": "Success",
   "data": [ ... ]
}
```

Invalid or Missing API Key:

```
HTTP/1.1 403 Forbidden
{
   "message": "Forbidden: Invalid API key"
}
```

6. Example: Browser CORS Failure

```
fetch("https://api-id.execute-api.region.amazonaws.com/prod/data", {
   method: "POST",
   headers: { "Content-Type": "application/json" },
}) body: JSON.stringify({ "message": "Hello" })
```

Browser Error (CORS):

Access to fetch at 'https://api-id.execute-api.region.amazonaws.com/prod/data' from origin 'https://example.com' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.

Fix: Allow the Origin in CORS Settings

Add the correct origin, methods, and headers in API Gateway.
After updating CORS, the browser request succeeds.

7. Troubleshooting Common Issues

Issue	Possible Cause	Solution
403 Forbiddon (Invalid API key)	Missing or incorrect API key	Include the correct x-api-key header and check the usage plan.
CORS error in browser	Missing or incorrect CORS headers	Verify allowed origins, methods, and headers in API Gateway.
Preflight request fails	OPTIONS method not allowed	Enable CORS and allow OPTIONS.
429 Too Many Requests	Rate limit or quota exceeded	Adjust throttling settings in the usage plan.
Unexpected 403	API key not associated with usage plan	Associate the API key with the correct usage plan and stage.

8. Practice This

1. Call the API with a valid API key.
2. Call the API with an invalid or missing API key.
3. Reproduce a browser CORS failure.
4. Fix the CORS configuration and test again.
5. Test throttling by sending multiple requests.

9. Production Insight

- Do not use API keys alone for authentication or authorization.
- Use authentication (e.g. JWT) for identity, throttling for protection and CORS for browser access control.
- Keep secrets out of source code, frontend code, and logs.
- REST API usage plans and API keys are supported, but AWS advises against using API keys alone for security.
- Design your API security using multiple layers.

Find the Failure: 400, 401, 403, 404, 429, 500, and 502

Focus: Stop guessing what an HTTP error means.

1. Categories of Failures

Client Errors (4xx)

The request from the client is invalid, unauthorized or cannot be fulfilled.

Authorization Errors

The client is not authenticated or does not have permission to access the resource.

Gateway Errors (429)

The request is rejected by API Gateway due to throttling or rate limits.

Backend Errors (5xx)

The request reached the backend but the server (Lambda or integration) failed to process it.

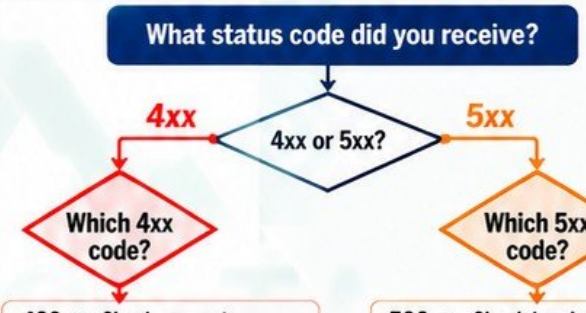
2. Common HTTP Error Codes

Status Code	Category	What It Means	Common Cause
400	Client Error	**Bad Request.** The request is malformed or fails validation.	Invalid JSON, missing fields, wrong format.
401	Authorization Error	**Unauthorized.** Authentication is missing or invalid.	Missing or invalid token, expired token.
403	Authorization Error	**Forbidden.** You are authenticated but do not have permission.	Insufficient role/scope, not an admin.
404	Client Error	**Not Found.** The resource or route does not exist.	Wrong URL, missing route, nonexistent resource.
429	Gateway Error	**Too Many Requests.** Rate limit exceeded.	Throttling, request burst, API limits.
500	Backend Error	**Internal Server Error.** An unexpected error in the backend.	Lambda error, unhandled exception, runtime error.
502	Gateway Error	**Bad Gateway.** API Gateway received an invalid response from the backend.	Lambda timeout, malformed response, integration issue.

3. Examples and How to Reproduce

Status Code	Example Request	What to Do
400	POST /tasks with invalid JSON	Send malformed JSON or missing required fields.
401	GET /profile without token	Do not include an Authorization header.
403	GET /admin/reports with normal user token	Use a valid token without the required admin scope.
404	GET /tasks/9999 (nonexistent resource)	Request a resource that does not exist.
429	Send many requests quickly	Use a loop or tool to send multiple requests to trigger throttling.
500	Force a Lambda error	Add code that throws an exception.
502	Return an invalid response	Return a malformed response from Lambda (e.g. not valid JSON).

4. Troubleshooting Decision Tree



4xx

- 400 → Check request format and validation.
- 401 → Check authentication token.
- 403 → Check user permissions and scopes.
- 404 → Check route and resource ID.
- 429 → Check throttling and rate limits.

5xx

- 500 → Check Lambda logs for errors.
- 502 → Check Lambda response format and integration.
- Check CloudWatch logs.
- Verify API Gateway integration settings.
- Confirm the backend is healthy and returning a valid response.

5. Sample Error Responses

400 Bad Request

```
{
  "message": "Invalid JSON body"
}
```

401 Unauthorized

```
{
  "message": "Missing or invalid token"
}
```

403 Forbidden

```
{
  "message": "User is not allowed to access this resource"
}
```

404 Not Found

```
{
  "message": "Task not found"
}
```

429 Too Many Requests

```
{
  "message": "Rate limit exceeded"
}
```

500 Internal Server Error

```
{
  "message": "Internal server error"
}
```

6. Practice This

1. Send a request with invalid JSON and identify the error.
2. Call a protected endpoint without a token (401).
3. Call an admin endpoint with a normal user token (403).
4. Request a nonexistent resource (404).
5. Send multiple requests quickly to trigger 429.
6. Introduce a Lambda error and observe 500 or 502.
7. For each failure, explain what happened and which component is responsible.

7. Troubleshooting Checklist

- ✓ Check the request format (headers, body, parameters).
- ✓ Verify authentication token (present, valid, not expired).
- ✓ Check user permissions and scopes.
- ✓ Confirm the route and resource ID are correct.
- ✓ Review API Gateway throttling settings.
- ✓ Check Lambda logs in CloudWatch.
- ✓ Verify the backend returns a valid JSON response.

8. Production Insight

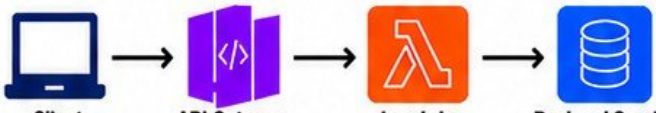
- Do not rely only on status codes; always check logs.
- Client errors (4xx) are usually caused by your request or permissions.
- Gateway errors (429, 502) are related to API Gateway or integration.
- Server errors (500) indicate a problem in your backend code.
- Good logging, clear error messages, and monitoring make troubleshooting faster.

Read CloudWatch Logs Like an Engineer

Focus: Follow one failed request through the system.

1. End-to-End Request Flow

A request goes through multiple components. To find the cause of a failure, follow it through each stage using logs.



Client (Send Request) → API Gateway (Access Logs) → Lambda (Application Logs) → Backend Service (e.g. DynamoDB, External API)

2. Enable API Gateway HTTP API Access Logging

1. Open your HTTP API in API Gateway.
2. Go to Stage → Logs and tracing.
3. Enable access logging and create a CloudWatch Log Group.
4. Choose a useful log format (JSON) that includes request ID, status, latency and other fields.
5. Deploy the stage.

Example Access Log Format (JSON)

```
{
  "requestId": "$context.requestId",
  "ip": "$context.identity.sourceIp",
  "requestTime": "$context.requestTime",
  "httpMethod": "$context.httpMethod",
  "routeKey": "$context.routeKey",
  "status": $context.status,
  "responseLength": $context.responseLength,
  "integrationLatency": $context.integrationLatency,
  "message": "$context.error.message"
}
```

3. Understand Key Log Sources

API Gateway Access Logs

- Records every request received by your API.
- Includes request ID, source IP, route, method, status code, and latency.
- Helps you see if the request reached your backend.

Lambda Application Logs

- Contains logs from your function code (print statements, logger output, and errors).
- Shows function execution time and any exceptions.
- Use the same request ID or custom correlation ID to connect with API Gateway logs.

4. Correlate API Gateway Logs With Lambda Logs

1. Find the request in API Gateway access logs using the request ID or timestamp.
2. Locate the corresponding Lambda log entry (same request ID if logged).
3. Compare timestamps to understand where the delay or failure happened.

Example Lambda Log With Request ID

```
{
    "timestamp": "2026-10-29T12:34:56Z",
    "level": "ERROR",
    "requestId": "a1b2c3d4-5678-90ab-cdef-11111111111111",
    "message": "Failed to get task from DynamoDB",
    "error": "ResourceNotFoundException: Requested resource not found"
}
```

5. Understanding Latency Fields

Field	Meaning	What It Tells You
integrationLatency	Time taken by API Gateway to get a response from Lambda.	High value means backend (Lambda) is slow.
responseLatency	Total time to return a response to the client.	Includes integration latency + other processing time.
lambda duration	Actual execution time of your function.	Found in Lambda logs ("Duration" field).

6. Example: Tracing a Failed Request

1. Send a request that will fail (e.g. invalid task ID).
2. Find the request in API Gateway access logs (look at status code and integrationLatency).
3. Locate the same request in Lambda logs using the request ID.
4. Check the error message and stack trace in Lambda logs.
5. Identify the failing component (API Gateway, Lambda, or backend).

7. Common Log Patterns and What They Mean

Log Pattern / Message	Where It Appears	Likely Cause
400 Bad Request	API Gateway	Invalid request format or missing fields.
401 Unauthorized	API Gateway	Missing or invalid token.
403 Forbidden	API Gateway or Lambda	Authenticated but not permitted.
404 Not Found	API Gateway or Lambda	Route or resource does not exist.
429 Too Many Requests	API Gateway	Throttling/rate limit exceeded.
500 Internal Server Error	Lambda logs	Unhandled exception in function code.
502 Bad Gateway	API Gateway	Lambda timeout or invalid response.

8. Using CloudWatch Logs Insights

1. Open CloudWatch → Logs Insights.
2. Select the log group (API Gateway or Lambda).
3. Use a query to find recent errors.
4. Filter by request ID, status code or time range.

Example Query (Find Errors)

```
fields @timestamp, @message
| filter @message like /ERROR/ or @message like /Exception/
| sort @timestamp desc
| limit 50
```

9. Security Best Practices for Logging

- Include useful request IDs and status fields.
- Do not log access tokens, passwords or sensitive request bodies.
- Sanitize or mask sensitive data (e.g. emails, personal information).
- Use structured JSON logs for easier searching.
- Set appropriate log retention periods.

10. Practice This

1. Make a request that will fail (e.g. invalid ID or missing field).
2. Find the logs in API Gateway and Lambda.
3. Identify the status code, latency and error message.
4. Explain which component failed and why.
5. Fix the issue and verify the request works.

11. Production Insight

- Use logs and metrics to understand what is happening, not guesses.
- Correlate logs using request IDs and timestamps.
- Distinguish gateway latency from backend execution time.
- Good logging and monitoring make troubleshooting faster and easier.

Troubleshoot Timeouts, Permissions, and Broken Integrations

Focus: Diagnose failures where API Gateway and Lambda do not communicate correctly.

1. How API Gateway Calls a Lambda Function



2. Understand the Two Types of Permissions

Lambda Resource-Based Invocation Permission

- Allows API Gateway to invoke your Lambda function.
- Configured as a resource policy on the Lambda function.
- Specifies the API Gateway principal and source ARN (API ID + stage + route).
- If missing, API Gateway returns 403 Forbidden.

Lambda Execution-Role Permissions

- Allows your Lambda function to access AWS services (e.g. DynamoDB, S3, CloudWatch).
- Configured using an IAM role attached to the function.
- If missing or incorrect, your function code fails with AccessDenied errors.
- This does not affect API Gateway invocation permission.

3. Common Integration Failures and Symptoms

Failure Type	HTTP Status	What It Means	Common Cause
Missing invocation permission	403	API Gateway is not allowed to invoke the Lambda function.	Lambda resource policy missing or incorrect source ARN.
Malformed Lambda proxy response	502	Lambda returned an invalid response format.	Missing statusCode, body not a string, wrong JSON structure.
Lambda timeout	504	Lambda did not return in time.	Function took longer than the configured timeout (default 29s).
Function error	502	Lambda threw an exception.	Unhandled error in function code.
Downstream service failure	500	Backend dependency failed.	DynamoDB, external API or other service error.

4. Lambda Proxy Response Format (Correct vs Incorrect)

Correct Response

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json"
  },
  "body": "{\"message\": \"Success\"}"
}
```

Incorrect Response

```
{
  "status": 200,
  "body": {
    "message": "Success"
  }
}
```

Missing statusCode and body is not a string.

5. Integration Timeout and Cold Starts

- API Gateway has a maximum integration timeout of 30 seconds for HTTP APIs.
- If your Lambda function takes longer than the timeout, API Gateway returns 504.
- Cold starts can increase initial response time, especially with larger packages.
- Timeouts are often caused by slow code or downstream dependencies (e.g. slow database queries or external APIs), not just Lambda itself.

6. Check CloudWatch Logs and Metrics

1. Open CloudWatch → Log groups.
2. Find the API Gateway access logs for the request.
3. Locate the corresponding Lambda log stream using the request ID.
4. Check for errors, exceptions, and duration.
5. Use CloudWatch Metrics to review invocations, errors, and duration.

Useful CloudWatch Logs Insights Query

```
fields @timestamp, @message, @logStream
| filter @message like /ERROR/ or @message like /Exception/
| sort @timestamp desc
| limit 50
```

7. Test and Reproduce a Permission Failure

1. Remove the Lambda resource-based permission for API Gateway.
2. Call the API and observe the 403 Forbidden response.
3. Check the API Gateway execution logs for the error message.
4. Add the correct permission back.
5. Call the API again and verify it works.

Example Error (Missing Permission)

```
{
  "message": "User: apigateway.amazonaws.com
   is not authorized to perform: lambda:InvokeFunction",
  "statusCode": 403
}
```

8. Troubleshooting Checklist

- ☑ Verify Lambda resource-based invocation permission.
- ☑ Check Lambda execution-role permissions.
- ☑ Validate Lambda proxy response format.
- ☑ Confirm integration timeout is sufficient (but not too high).
- ☑ Inspect CloudWatch logs for detailed error messages.
- ☑ Check for cold starts and downstream service delays.
- ☑ Verify that environment variables and configuration are correct.
- ☑ Test the endpoint after making each change.

9. Production Insight

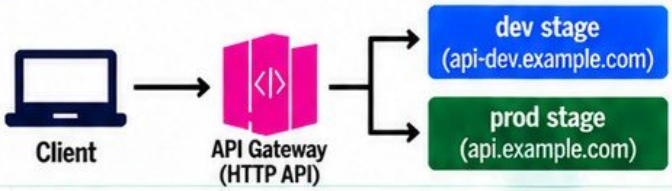
- A 403 error usually means a permission problem between API Gateway and Lambda.
- A 502 error often means your Lambda function returned an invalid response or threw an unhandled exception.
- A 504 error means a timeout, which is usually caused by slow code or a slow downstream dependency.
- Increasing the timeout does not fix a slow database or broken service.
- Always use logs and metrics to identify the real cause.

Deploy Changes Without Breaking Existing Clients

Focus: Treat an API as a service that other applications depend on.

1. Stages and Deployments in API Gateway

- A stage represents an environment (e.g. dev, prod).
- A deployment is a snapshot of your API configuration.
- HTTP APIs can use automatic deployment or manual deployment.
- Changes to routes, integrations or authorizers must be deployed to a stage.
- Use separate stages for development and production.



2. Development vs Production Environments

Development (dev)

- Used for testing and iteration.
- Can use separate resources (e.g. dev database).
- More frequent deployments.
- May have relaxed settings.

Production (prod)

- Used by real users and clients.
- Stable and carefully tested changes.
- Stronger security and monitoring.
- Separate configuration and environment variables.

3. Breaking vs Backward-Compatible Changes

Breaking Changes (avoid for existing clients)

- Removing existing endpoints.
- Changing response format in an incompatible way.
- Renaming fields.
- Changing required parameters.
- Changing authentication requirements.

Backward-Compatible Changes (safer)

- Adding new endpoints.
- Adding optional fields in the response.
- Supporting both old and new fields during transition.
- Using versioning (e.g. /v1).
- Keeping existing fields and behavior unchanged.

4. API Versioning with /v1

- Versioning helps maintain contract stability.
- Use a versicte in the path (e.g. /v1/tasks).
- You can introduce /v2 for new, incompatible changes.
- Keep older versions running until clients migrate.

Example Routes

```
GET     /v1/tasks
POST    /v1/tasks
PATCH   /v1/tasks/{taskId}
DELETE  /v1/tasks/{taskId}
GET     /v2/tasks   (new version with changes)
```

5. Document API Routes with OpenAPI

- Use OpenAPI (Swagger) to document your API.
- Include endpoints, request/response schemas, and examples.
- Helps developers understand how to use your API.
- Keep the documentation updated when you make changes.

Example OpenAPI Snippet

```yaml
openapi: 3.0.1
info:
  title: Task API
  version: 1.0.0
paths:
  /v1/tasks:
    get:
      summary: List tasks
      responses:
        '200':
          description: Successful response
```

6. Deploy Infrastructure with AWS SAM

- Use AWS SAM (or Terraform/CDK) to define and deploy your API.
- Keep infrastructure as code for repeatable deployments.
- Define stages, environment variables, and permissions in your template.
- Deploy to dev and prod using different parameters.

Example SAM Snippet

```yaml
Resources:
  Api:
    Type: AWS::Serverless::HttpApi
    Properties:
      StageName: prod
      AutoDeploy: true
      CorsConfiguration:
        AllowOrigins: ["https://yourdomain.com"]
```

7. Test Before Release

- Test successful and failed responses.
- Verify that existing clients still work.
- Check different scenarios (valid, invalid, unauthorized).
- Use a separate dev stage for testing changes.

8. Example: Change a Response Safely

Before (v1 response)

```
{
  "id": "123",
  "title": "Test Task",
  "status": "pending"
}
```

After (v1 with new optional field)

```
{
  "id": "123",
  "title": "Test Task",
  "status": "pending",
  "updatedAt": "2026-10-29T12:00:00Z"
}
```

9. Troubleshooting Common Issues

Issue	Possible Cause	Solution
Wrong stage URL	Using dev instead of prod (or vice versa)	Check the correct stage endpoint.
Changes not visible	Not deployed to the stage	Deploy the API or enable automatic deployment.
Incorrect environment variables	Wrong configuration for the stage	Verify stage variables and Lambda environment variables.
Clients breaking	Breaking change deployed	Use backward-compatible changes or introduce a new version (/v2).

10. Practice This

1. Update an API response by adding a new optional field.
2. Deploy the changes to a dev stage.
3. Verify existing endpoints still work.
4. Test with both valid and invalid requests.
5. Deploy to a prod stage (or simulate) and confirm compatibility.

11. Production Insight

- Always treat your API as a contract with other applications.
- Use versioning and backward-compatible changes.
- Test thoroughly before releasing to production.
- A deployment can introduce issues if the wrong configuration or environment variables are used.
- Automate deployments and keep clear separation between environments.

Monitor the APIs Before Users Report Failures

Focus: Turn ten working APIs into an observable service.

1. What to Monitor

1. Request volume (number of requests).
2. 4XX errors (client errors).
3. 5XX errors (server errors).
4. Latency (response time).
5. Dependency health (e.g. DynamoDB, external APIs).
6. Resource limits and throttling.
7. Trends over time (detect unusual patterns).



2. API Gateway Metrics vs Lambda Metrics

Metric Type	API Gateway Metrics	Lambda Metrics
Purpose	Shows how the API is being used at the edge.	Shows how your function is running.
Key Metrics	RequestCount, 4XXError, 5XXError, Latency, IntegrationLatency, Count by route and stage.	Invocations, Errors, Duration, Throttles, ConcurrentExecutions.
What it Tells You	Client traffic, error rates, edge latency, and integration issues.	Function execution time, errors, and resource constraints.

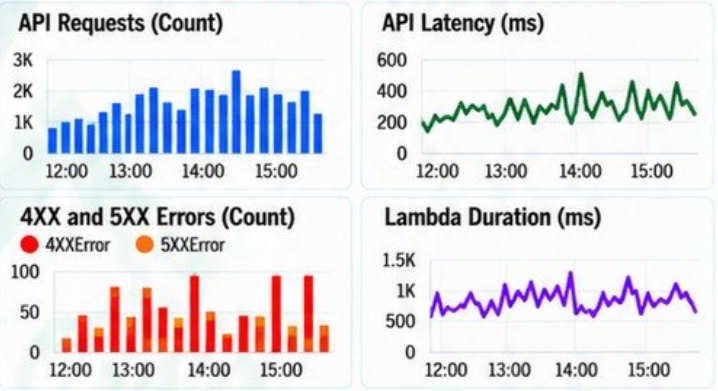
3. Example Metrics to Track

Metric	Service	What It Means
RequestCount	API Gateway	Total number of requests.
4XXError	API Gateway	Client errors (e.g. 400, 401, 403).
5XXError	API Gateway	Server errors (e.g. 500, 502).
Latency	API Gateway	Total response time from client perspective.
IntegrationLatency	API Gateway	Time taken to complete integration with Lambda.
Invocations	Lambda	Number of times your function ran.
Errors	Lambda	Function errors.
Duration	Lambda	Execution time of the function.
Throttles	Lambda	Requests throttled due to limits.
ThrottledRequests	DynamoDB	Requests throttled due to table limits.

4. Create a CloudWatch Dashboard

1. Open CloudWatch → Dashboards → Create dashboard.
2. Add API Gateway metrics: RequestCount, 4XXError, 5XXError, Latency.
3. Add Lambda metrics: Invocations, Errors, Duration, Throttles.
4. Add DynamoDB metrics: ConsumedReadCapacity, ConsumedWriteCapacity, ThrottledRequests.
5. Arrange widgets to show a clear view of your APIs.

Example Dashboard (Key Widgets)



5. Set a Useful Alarm

1. Choose a meaningful metric (e.g. 5XXError > threshold).
2. Set the threshold and evaluation period.
3. Configure an action (e.g. SNS notification to email/Slack/Telegram).
4. Test the alarm to confirm it works.

Example Alarm Configuration

Metric: 5XXError (API Gateway)
Threshold: >= 10 errors
Period: 5 minutes
Action: Send notification via SNS (email/Slack/Telegram)

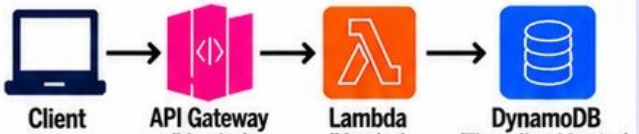


6. High Traffic vs Real Incident

High Traffic (Not an Incident)	Real Incident
Increase in RequestCount.	Sudden increase in 5XX errors.
Latency may be slightly higher but still within normal range.	High latency or timeouts.
Error rates (4XX and 5XX) remain low.	Lambda errors or throttling.
Usually expected (e.g. marketing event or new feature).	DynamoDB throttling.
	Service dependency failures.
	Users cannot complete key actions.

7. Dependency Health and DynamoDB Throttling

- Monitor DynamoDB ConsumedReadCapacity and ConsumedWriteCapacity.
- Watch ThrottledRequests for signs of capacity issues.
- Check external API dependencies for high latency or failures.
- Use CloudWatch metrics and logs to identify the root cause.



8. Practice This

1. Send normal traffic to your APIs and observe the metrics.
2. Generate a controlled failure (e.g. return 500 from Lambda).
3. Check the dashboard to see the effect on error metrics and latency.
4. Verify that the alarm triggers.
5. Identify which component is failing (API Gateway, Lambda, or DynamoDB).
6. Restore the service and confirm metrics return to normal.



9. Production Insight



- A dashboard shows system behavior: it does not fix problems.
- A useful alert requires someone to respond.
- Monitor both your APIs and their dependencies.
- Focus on meaningful metrics and avoid alerting on normal traffic spikes.
- Good monitoring helps you detect and resolve issues before users report them.

Run a Realistic API Incident

Scenario: TaskFlow users suddenly cannot create tasks.

1. Incident Overview

- **Service:** TaskFlow
- **Affected endpoint:** POST /tasks
- **Initial symptom:** 5XX responses
- **Impact:** Users cannot create tasks
- **Goal:** Investigate, fix, and write an incident report using evidence.

POST /tasks
5XX Error

2. Investigation Steps

1. Confirm the issue and reproduce the error.
2. Check API Gateway access logs.
3. Check Lambda errors and execution metrics.
4. Compare the failure time with recent deployments.
5. Investigate DynamoDB permissions and integration behavior.
6. Form and test a hypothesis.
7. Apply a controlled fix or rollback.
8. Verify successful task creation.

3. Reproduce the Issue

```
curl -X POST https://api.example.com/tasks \\
  -H "Content-Type: application/json" \
  -d '{"title": "Test task"}'
```

```
HTTP/1.1 500 Internal Server Error
{
    "message": "Internal server error"
}
```

4. Check API Gateway Access Logs

```
{
  "requestId": "a1b2c3d4-5678-90ab-cdef",
  "httpMethod": "POST",
  "routeKey": "POST /tasks",
  "status": 500,
  "integrationErrorMessage": "Execution failed
 due to an internal error",
  "integrationLatency": 1234
}
```

5. Check Lambda Logs and Metrics

```
START RequestId: a1b2c3d4-5678-90ab-cdef
ERROR  AccessDeniedException: User is not authorized
to perform: dynamodb:PutItem  on resource:
arn:aws:dynamodb:...
END RequestId: a1b2c3d4-5678-90ab-cdef
REPORT RequestId: a1b2c3d4   Duration: 250.12 ms
```

Key Metrics to Review (CloudWatch)

- Errors (Lambda)
- Invocations
- Duration
- Throttles
- 5XXError (API Gateway)
- IntegrationLatency (API Gateway)
- DynamoDB ThrottledRequests
- DynamoDB ConsumedCapacity

6. Compare With Recent Deployments

- Check the time of the failure.
- Review recent code or infrastructure changes.
- Look for changes to IAM roles, environment variables, or DynamoDB configuration.
- Identify what changed before the incident started.

7. Form and Test a Hypothesis

- Example hypothesis: The Lambda execution role no longer has permission to write to DynamoDB.
- Check the IAM policy attached to the execution role.
- Test by adding the required permission in a controlled environment.
- Verify whether the error is resolved.

8. Apply a Fix or Rollback

1. Update the Lambda execution role with the correct DynamoDB permissions, or
2. Roll back to the previous working version.
3. Deploy the change.
4. Test the endpoint again.

9. Verify the Fix

```
curl -X POST https://api.example.com/tasks  \
  -H "Content-Type: application/json" \
  -d '{"title": "Task created"}'
```

```
HTTP/1.1 201 Created
{
    "id": "123",
    "title": "Task created"
}
```

10. Incident Report (Example Structure)

- **Impact:** What users experienced and how many were affected.
- **Timeline:** Key events with timestamps (detection, investigation, fix).
- **Root Cause:** Why the incident happened.
- **Resolution:** What was done to fix it.
- **Prevention:** What will be changed to avoid it in the future.

11. Final Challenge

- Explain why the incident happened, not just which command fixed it.
- Use evidence from logs, metrics, and configuration changes.
- Describe how you would prevent a similar incident in the future.

Final Engineering Challenge: Prove All 10 APIs Work

Focus: Complete an end-to-end technical verification of the TaskFlow service.

1. The 10 API Endpoints to Verify

#	Method	Endpoint	Purpose
1	POST	/auth/register	Register a new user
2	POST	/auth/login	Authenticate and get token
3	GET	/tasks	List all tasks (user's tasks)
4	POST	/tasks	Create a new task
5	GET	/tasks/{taskId}	Get a specific task
6	PUT	/tasks/{taskId}	Update a task
7	DELETE	/tasks/{taskId}	Delete a task
8	POST	/feedback	Submit feedback (rate limited)
9	GET	/admin/reports	Admin-only reports (requires admin scope)
10	GET	/health	Service health check

2. Test All Endpoints with curl or an API Client

1. Use curl, Postman, or any API client.
2. Test both successful and failed requests.
3. Include authentication tokens where required.
4. Verify correct status codes and response bodies.
5. Save your test results (screenshots or terminal output).

Example: Test Create Task

```
curl -X POST https://api.example.com/tasks \
  -H "Authorication: Bearer <ACCESS_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"title": "Buy groceries", "completed": false}'
```

```
HTTP/1.1 201 Created
{
   "id": "123",
   "title": "Buy groceries",
   "completed": false
}
```

3. Verify Success and Failure Scenarios

Scenario	Expected Result	Example Status Code
Valid request (authorized)	Success	200, 201
Missing or invalid token	Unauthorized	401
Insufficient permissions	Forbidden	403
Validation error (bad input)	Bad Request	400
Nonexistent resource	Not Found	404
Rate limit exceeded	Too Many Requests	429
Server or integration error	Internal Server Error	500
Bad gateway / downstream error	Bad Gateway	502

4. Authentication and Authorization Checks

- Test endpoints with and without authentication.
- Verify user tokens can only access their own resources.
- Verify admin endpoints require the correct scope/role.
- Confirm that normal users are denied access to admin endpoints.
- Check that sensitive data is not exposed in responses or logs.

5. Test Core Task Operations

1. Create a task.
2. Retrieve the task by ID.
3. List all tasks.
4. Update the task.
5. Delete the task.
6. Verify data is stored correctly in DynamoDB.

6. Check Validation, Rate Limiting, and Error Handling

- Send invalid payloads and confirm validation errors (400).
- Request a nonexistent task and confirm 404.
- Send repeated requests to /feedback and confirm rate limiting (429).
- Test error handling for downstream service failures (500/502).

7. Inspect Logs and Metrics

- Check API Gateway access logs for successful and failed requests.
- Check Lambda logs for execution details and errors.
- Confirm the request IDs match between API Gateway and Lambda.
- Review CloudWatch metrics for request count, 4XX, 5XX and latency.
- Ensure alarms are configured and working.

8. Review IAM Permissions and Security

- Verify Lambda execution roles follow least privilege.
- Confirm API Gateway has permission to invoke Lambda.
- Check DynamoDB permissions are correct and limited.
- Ensure no secrets (tokens, passwords, keys) are in code or logs.
- Review environment variables and configuration.

9. Document the API Service

- Write clear endpoint documentation (OpenAPI or README).
- Include request/response examples.
- Document authentication requirements and scopes.
- Provide deployment instructions (e.g. AWS SAM).
- Include troubleshooting steps and common errors.

10. Clean Up Unused Resources

- Remove unused test resources, stacks, or tables.
- Review IAM roles and policies for unused permissions.
- Clean up old log groups, temporary stages, and sample data.
- Verify your final environment is clean and intentional.

11. Final Deliverable

- A working TaskFlow API service with all 10 endpoints.
- Verified with reproducible tests (success and failure).
- Proper authentication, authorization, validation and rate limiting.
- Operational visibility with logs, metrics and alarms.
- Clear documentation and deployment instructions.
- Security-conscious implementation with least privilege.

12. Final Challenge

- Explain why the incident happened, not just which command fixed it.
- Use evidence from logs, metrics, and configuration.
- Show that you understand the system behavior.
- Demonstrate that the APIs are reliable, secure, and observable.
- Be able to reproduce the tests on a clean environment.